

TOC - form 37 (Only Numericals)

37. ① concatenation

Suppose $x = ab$, $y = ba$
 $xy = abba$

② length $|abba| = 4$, $|a| = 1$, $|\epsilon| = 0$

③ Reverse $w^R = \text{Reverse of the string } w$ $(abba)^R = abba$
 $(abc)^R = cba$

$$L_1 = \{a, ab\}, L_2 = \{b, ba\}$$

$$(i) L_1 \cup L_2 = \{a, ab, b, ba\}$$

$$(ii) L_1 \cdot L_2 = \{a, ab\} \cdot \{b, ba\}$$

$$= \{ab, aba, abb, abba\}$$

Rice's Theorem

Statement: Every non-trivial property of recursively enumerable languages accepted by Turing Machines is undecidable.

Consider the property:

$$P = \{M \mid L(M) = \emptyset\}$$

("The language accepted by TM M is empty.")

- This property depends only on the language recognized by the TM.
- It is **non-trivial** because:
 - Some TMs accept no strings ($L(M) = \emptyset$).
 - Some TMs accept at least one string ($L(M) \neq \emptyset$).

Since P is a non-trivial property of recursively enumerable languages, **Rice's Theorem applies.**

Hence,

$$E_{TM} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

is **undecidable**.

Conclusion: By Rice's Theorem, determining whether the language accepted by a Turing Machine is empty is **undecidable**.

38, 39, 40 $\rightarrow$ Theory.

41 $\rightarrow$ Q.25

42. $S \rightarrow Sa|b$

the grammar has direct left recursion b/c the production has S as the leftmost symbol.

General rule

grammar form

$$A \rightarrow A\alpha | \beta$$

Replace

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' | \epsilon \end{aligned}$$

Applying Rule -

$$\begin{aligned} A &= S \\ \alpha &= a \\ \beta &= b \end{aligned}$$

$\therefore$ ans

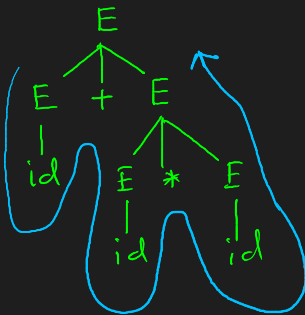
$$\begin{aligned} S &\rightarrow bS' \\ S' &\rightarrow aS' | \epsilon \end{aligned}$$

Ans

43. $E \rightarrow E + E \mid E * E \mid id$

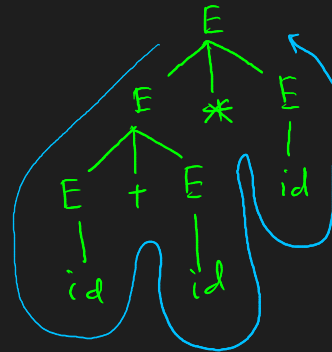
consider string id + id * id

① Parse Tree 1: id + (id * id)



$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow id + E \\ &\Rightarrow id + id * id \end{aligned}$$

② Parse Tree 2: (id + id) * id



$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow E + E * E \\ &\Rightarrow id + id * id \end{aligned}$$

String same: id + id * id

id + (id * id) (id + id) * id.

$\therefore$ Ambiguous !

44.

Parse Tree

A **parse tree** (derivation tree) is a tree representation of how a string is derived from the start symbol of a CFG.

- Root node = Start symbol
- Internal nodes = Non-terminals
- Leaf nodes = Terminals (or ϵ)
- Reading leaves from left to right gives the generated string.

52.

Yes.

An ϵ -NFA is an NFA that may have ϵ -transitions. Since ϵ -transitions are **optional**, an NFA can be viewed as an ϵ -NFA with **no ϵ -transitions**.

Therefore,

Every NFA is an ϵ -NFA, but not every ϵ -NFA is an NFA.

NFA over $\Sigma = \{0,1\}$ that accepts $|w| \geq 2$

State Diagram

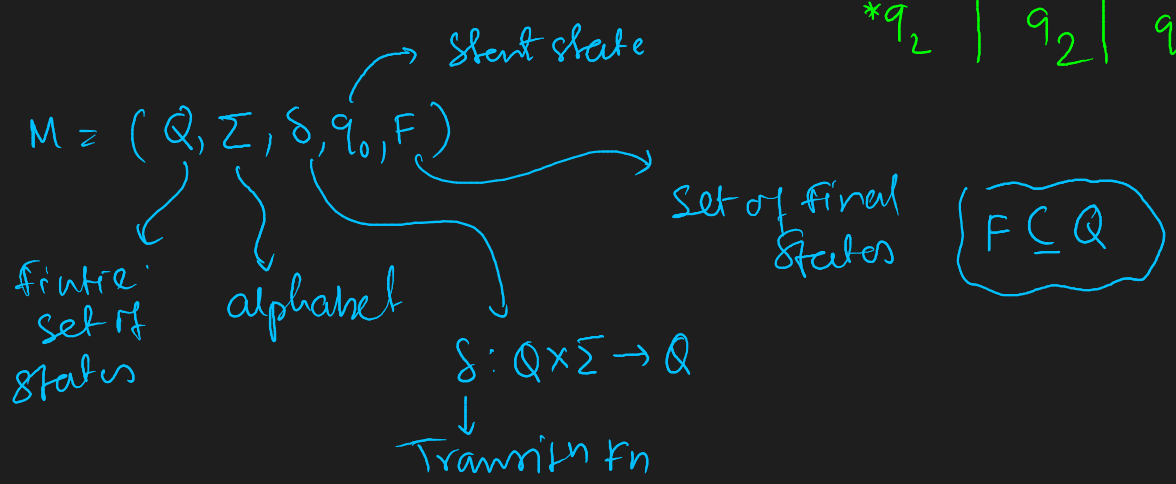


$q_0 \rightarrow \text{Start}$
 $Q = \{q_0, q_1, q_2\}$
 $F = q_2$

Transition fn

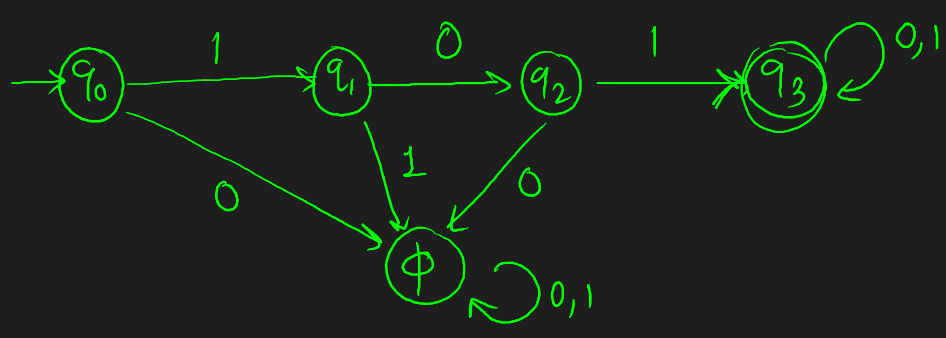
State	0	1
$\rightarrow q_0$	q_1	q_1
q_1	q_2	q_2
$*q_2$	q_2	q_2

Sol.



DFA

$L = \{w \in \{0,1\}^* \mid w \text{ begins with } 101\}$
 $\{ \underline{101}, \underline{101}0, \underline{101}011, \underline{1011}01, \dots \}$



State	0	1
q_0	ϕ	q_1
q_1	q_2	ϕ
q_2	ϕ	q_3
q_3	q_3	q_3
ϕ	ϕ	ϕ

90. $P = \{S \rightarrow aSb, S \rightarrow ab\}$

Base case: $S \Rightarrow ab$ string = ab

One Recursion: $S \Rightarrow aSb$
 $\Rightarrow a(ab)b$
 $\Rightarrow aabb$

Two Recursion: $S \Rightarrow aSb$
 $\Rightarrow aaSbb$
 $\Rightarrow aaaSbbb$

$\therefore$ Strings generated: $\{ab, aabb, aaabbb, \dots\}$

Language: $L = \{a^n b^n \mid n \geq 1\}$, $S \rightarrow aSb \mid ab$

91. $\rightarrow$ same as Q.6; Q.25

92. $S \rightarrow aA \mid b$
 $A \rightarrow aS \mid bA$

NFA transition table

State	a	b
$\rightarrow S$	A	F
A	S	A
*F	-	-

Convert to
DFA

State	a	b
$\rightarrow \{S\}$	$\{A\}$	$\{F\}$
$\{A\}$	$\{S\}$	$\{A\}$
$\{F\}$	ϕ	ϕ
ϕ	ϕ	ϕ

Final
Table

State	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_0	q_1
* q_2	ϕ	ϕ
ϕ	ϕ	ϕ

